

Foundations of Monte Carlo tree search

[Monte Carlo tree search]

1.0 Content Block

Improvements Various modifications of the basic Monte Carlo tree search method have been proposed to shorten the search time. Some employ domain-specific expert knowledge, others do not. Monte Carlo tree search can use either light or heavy playouts. Light playouts consist of random moves while heavy playouts apply various heuristics to influence the choice of moves. These heuristics may employ the results of previous playouts (e.g. the Last Good Reply heuristic) or expert knowledge of a given game. For instance, in many Go-playing programs certain stone patterns in a portion of the board influence the probability of moving into that area. Paradoxically, playing suboptimally in simulations sometimes makes a Monte Carlo tree search program play stronger overall.

2.0 Content Block

Pure Monte Carlo game search This basic procedure can be applied to any game whose positions necessarily have a finite number of moves and finite length. For each position, all feasible moves are determined: k random games are played out to the very end, and the scores are recorded. The move leading to the best score is chosen. Ties are broken by fair coin flips. Pure Monte Carlo Game Search results in strong play in several games with random elements, as in the game EinStein würfelt nicht!. It converges to optimal play (as k tends to infinity) in board filling games with random turn order, for instance in the game Hex with random turn order. DeepMind's AlphaZero replaces the simulation step with an evaluation based on a neural network.

3.0 Content Block

Leaf parallelization, i.e. parallel execution of many playouts from one leaf of the game tree. **Root parallelization**, i.e. building independent game trees in parallel and making the move basing on the root-level branches of all these trees. **Tree parallelization**, i.e. parallel building of the same game tree, protecting data from simultaneous writes either with one, global mutex, with more mutexes, or with non-blocking synchronization.

4.0 Content Block

Monte Carlo method The Monte Carlo method, which uses random sampling for deterministic problems which are difficult or impossible to solve using other approaches, dates back to the 1940s. In his 1987 PhD thesis, Bruce Abramson combined minimax search with an expected-outcome model based on random game playouts to the end, instead of the usual static evaluation function. Abramson said the expected-outcome model "is shown to be precise, accurate, easily estimable, efficiently calculable, and domain-independent." He experimented in-depth with tic-tac-toe and then with machine-generated evaluation functions for Othello and chess. Such methods were then explored and successfully applied to heuristic search in the field of automated theorem proving by W. Ertel, J. Schumann and C. Suttner in 1989, thus improving the exponential search times of uninformed search algorithms such as e.g. breadth-first search, depth-first search or iterative deepening. In 1992, B. Brügmann employed it for the first time in a Go-playing program. In 2002, Chang et al. proposed the idea of "recursive rolling out and backtracking" with "adaptive" sampling choices in their Adaptive Multi-stage Sampling (AMS) algorithm for the model of Markov decision processes. AMS was the first work to explore the idea of UCB-based exploration and exploitation in constructing sampled/simulated (Monte Carlo) trees and was the main seed for UCT (Upper Confidence Trees).

5.0 Content Block

Bibliography Cameron Browne; Edward Powley; Daniel Whitehouse; Simon Lucas; Peter I. Cowling; Philipp Rohlfshagen; Stephen Tavener; Diego Perez; Spyridon Samothrakis; Simon Colton (March 2012). "A Survey of Monte Carlo Tree Search Methods". *IEEE Transactions on Computational Intelligence and AI in Games*. 4 (1): 1–43. Bibcode:2012ITCIA...4A6810B. CiteSeerX 10.1.1.297.3086. doi:10.1109/tciaig.2012.2186810. S2CID 9316331. Maciej ■wiechowski, Konrad Godlewski, Bartosz Sawicki, Jacek Ma■dziuk (July 2022). "Monte Carlo Tree Search: a Review of Recent Modifications and Applications". *Springer Nature Artificial Intelligence Review*. 56 (3): 497–2562 (66 pages). arXiv:2103.04931. doi:10.1007/s10462-022-10228-y.{{cite journal}}: CS1 maint: multiple names: authors list (link)

6.0 Content Block

w_i stands for the number of wins for the node considered after the i -th move n_i stands for the number of simulations for the node considered after the i -th move N_i stands for the total number of simulations after the i -th move run by the parent node of the one considered c is the exploration parameter—theoretically equal to $2^{\frac{1}{2}}$; in practice usually chosen empirically The first component of the formula above corresponds to exploitation; it is high for moves with high average win ratio. The second component corresponds to exploration; it is high for moves with few simulations. Most contemporary implementations of Monte Carlo tree search are based on some variant of UCT that traces its roots back to the AMS simulation optimization algorithm for estimating the value function in finite-horizon Markov Decision Processes (MDPs) introduced by Chang et al. (2005) in *Operations Research*. (AMS was the first work to explore the idea of UCB-based exploration and exploitation in constructing sampled/simulated (Monte Carlo) trees and was the main seed for UCT.)

7.0 Content Block

In 2006, inspired by its predecessors, Rémi Coulom described the application of the Monte Carlo method to game-tree search and coined the name Monte Carlo tree search, L. Kocsis and Cs. Szepesvári developed the UCT (Upper Confidence bounds applied to Trees) algorithm, and S. Gelly et al. implemented UCT in their program MoGo. In 2008, MoGo achieved dan (master) level in 9×9 Go, and the Fuego program began to win against strong amateur players in 9×9 Go. In January 2012, the Zen program won 3:1 in a Go match on a 19×19 board with an amateur 2 dan player. Google Deepmind developed the program AlphaGo, which in October 2015 became the first Computer Go program to beat a professional human Go player without handicaps on a full-sized 19x19 board. In March 2016, AlphaGo was awarded an honorary 9-dan (master) level in 19×19 Go for defeating Lee Sedol in a five-game match with a final score of four games to one. AlphaGo represents a significant improvement over previous Go programs as well as a milestone in machine learning as it uses Monte Carlo tree search with artificial neural networks (a deep learning method) for policy (move selection) and value, giving it efficiency far surpassing previous programs. The MCTS algorithm has also been used in programs that play other board games (for example Hex, Havannah, Game of the Amazons, and Arimaa), real-time video games (for instance Ms. Pac-Man and Fable Legends), and nondeterministic games (such as skat, poker, Magic: The Gathering, or Settlers of Catan).